ISCTF2025 WriteUp

by zijeff

Ez_Caesar

凯撒密码的变种，每次偏移量递增。仿照题目逻辑写一下就行，代码如下：

```
def variant_caesar_encrypt(text):
    encrypted = ""
    shift = 2
    for char in text:
        if char.isalpha():
            if char.isupper():
                base = ord('A')
                new_char = chr((ord(char) - base - shift) % 26 + base)
            else:
                base = ord('a')
                new_char = chr((ord(char) - base - shift) % 26 + base)
            encrypted += new_char
            shift += 3
        else:
            encrypted += char
    return encrypted

text = "KXKET{Tubsdx_re_hg_zytc_hxq_vnjma}"
print(variant_caesar_encrypt(text))
```

运行结果如图：

```
smith@jeff-ciallo:~/Tasks/ISCTF2025$ /bin/python3 /home/smith/Tasks/ISCTF2025/Ez_Caesar/exp.py
ISCTF{Caesar_is_so_easy_and_funny}
smith@jeff-ciallo:~/Tasks/ISCTF2025$
```

Power tower

读完代码之后，我们求解的关键在于计算出 l ，根据代码的生成过程，我们是无法直接计算的。

理由如下：

$$l = 2^e \mod n$$
$$e = 2^t$$

所以指数部分是非常大的，计算机内存不够，进而我们必须想办法化简这个形式。如何化简呢？我们利用一下**扩展欧拉定理**就可以了，证明过程就不多赘述了。

代码如下：

```
from Crypto.Util.number import *
```

```
t = 6039738711082505929
n =
1075029458432512443375350824606975836393574730160052520082628654811383550
40617
c =
1140928178886101840613065681774740336487379363261430992572508075290882135
65247

p = 127
q = 841705194007
r = 1005672644717572752052474808610481144121914956393489966622615553
phi = (p-1)*(q-1)*(r-1)

e = pow(2,t,phi)
print(e)
l = pow(2,e,n)
print(l)
flag = c ^ l
print(long_to_bytes(flag))
```

运行结果如图：

```
smith@jeff-ciallo:~/Tasks/ISCTF2025$ /bin/python3 /home/smith/Tasks/ISCTF2025/Power_tower/exp.py
636287895840905585954655980911969280767202832863838002043681884487727620915 20
820620698661798770892674778269186882120743227516516815206253097110267092414 10
b'ISCTF{Euler_1s_v3ry|useful!!!!!}'
smith@jeff-ciallo:~/Tasks/ISCTF2025$
```

baby_equation

多元高次方程组，在 sagemath 中直接求解是显然不太现实的了，求解速度太慢了。显然，我们要通过数学手段化简一下。首先，我们已知：

$$4b^6 - 2a^3 + 3ac = N_1$$
$$b^5 + 6c^3 + 2abc = N_2$$
$$3a^3 - 3ac - 3b^6 = N_3$$

接着，我们会很自然地将一式与三式叠加消去交叉项：

$$a^3 + b^6 = N_1 + N_3 = S$$
$$a^3 = S - b^6$$

因为直接求解是不太现实的，我们最好能根据数量上的关系初步确定未知量的范围，进而求解。观察后发现 N_1和 N_3 是很接近互为相反数的，由于高位很多位数是一致的。因此 S 相对于来说是一个小的数，所以：

$$a^3 \approx -b^6$$

$$a \approx -b^2$$

$$4b^6 - 2a^3 + 3ac = 4b^6 - 2(S - b^6) + 3ac = 6b^6 + 3ac - 2S = N_1$$

综上，b^6 在式子中占据了较大的权重，所以我们近似认为：

$$6b^6 \approx N_1$$

最后，在近似值附近搜索即可，确定 a, b 后即可解出 c 。

具体代码如下：

```
from Crypto.Util.number import *
import gmpy2

# 题目给出的常量
K1 =
553034660032333988523282054579841849962513278686939363642019712460600549007804150576591812076929393639560967570419719747986618629768646813390664025639091979945370189438299222312737437421258649226366128728795414341712895829850346444 8
K2 =
367238756648163493263214707316273668476850247269131667264181091565884300988892769135631899967878660649894960382858200404021324858223969613524595648258694286191117042361183398621750643518660662218141806549694988772288699959699911475779235 7
K3 =
-553034660032333988523282054579841849962513278686939363642019703556680506206453450370497675646831988865044166882636398484432720605642443975272628386202604241092119739637083923356070888600688456996993274961583807024392286637134591011 1

def solve():
    # 1. 计算 S = a^3 + b^6
    S = K1 + K3

    # 2. 估算 b
    # 由 6b^6 + 3ac - 2S = K1 且 6b^6 占主导地位
    # b 约等于 (K1 / 6) 的 6 次方根
    b_approx = gmpy2.iroot(K1 // 6, 6)[0]

    # 在估算值附近搜索 b
    # 因为忽略了项 3ac (负值) 和 -2S，实际的 6b^6 会比 K1 略大，所以 b >=
b_approx
    for delta in range(0, 5):
        b = int(b_approx + delta)

        # 3. 计算 a
        # a^3 = S - b^6
```

```python
        a3 = S - b**6

        # 判定是否为完全立方数
        if a3 < 0:
            a_val, exact = gmpy2.iroot(-a3, 3)
            if exact:
                a = -int(a_val)
            else:
                continue
        else:
            a_val, exact = gmpy2.iroot(a3, 3)
            if exact:
                a = int(a_val)
            else:
                continue

        # 4. 计算 c
        # 3ac = K1 - 4b^6 + 2a^3
        numerator = K1 - 4 * (b**6) + 2 * (a**3)
        denominator = 3 * a

        if numerator % denominator == 0:
            c = numerator // denominator

            # 5. 验证方程2 (可选，确保正确)
            # b^5 + 6c^3 + 2abc = K2
            lhs2 = b**5 + 6*(c**3) + 2*a*b*c
            if lhs2 == K2:
                print(f"[+] Found valid solution!")
                print(f"a = {a}")
                print(f"b = {b}")
                print(f"c = {c}")
                try:
                    flag = long_to_bytes(c)
                    print(f"[*] Flag: {flag.decode()}")
                except:
                    print(f"[*] Raw bytes: {long_to_bytes(c)}")
                return

    print("[-] Solution not found in range.")

if __name__ == "__main__":
    solve()
```

运行结果如图:

```
[+] Found valid solution!
a = -97319611529501810510904538298668204056042623868316550440771307534558768612892
b = 311960913464334198969500852124413736815
c = 849048344115672619127152622191605369187870416381505572905452188371370495850585 3
[*] Flag: ISCTF{y0u_93t_7h3_3qu4710n_50lv3}
```

easy_RSA

结合提示知道，这道题我们要用上欧拉定理了，过程如下：

$$
\begin{gathered}
c_2 = m^{p+q} \mod N \\
\varphi(N) = (p-1)(q-1) = pq - (p+q) + 1 = N - (p+q) + 1 \\
p + q = N - \varphi(N) + 1 \\
c_2 \equiv m^{N-\varphi(N)+1} \mod N \\
m^{\varphi(N)} \equiv 1 \mod N \\
c_2 \equiv m^{N+1} \mod N
\end{gathered}
$$

又因为：

$$
c_1 = m^e \mod N
$$

所以，我们使用共模攻击即可，具体代码如下：

```python
from Crypto.Util.number import *
from sage.all import xgcd

N =
176302582570805577970623204744235159677059500264150129120876556793154791 6
890398090172842514078700504603800006841426993680647882826084885975340078 6
557270120330760791255046985114127285672634413513991988895166115794242018 6
740425637883483815675651901462780408112577571190902964786107983939445818 7
030937352988495066399048552564620003422064890149083596296402993632115520 0
390798215987316069871958913773199197073860062515329879288106446016695204 4
260013935663515240238573329782608944096985964654742148984027071579333264 3
189662902519796420958099182122255766358947558942303213099345652217854045 5
360695933336455068507071827928617
ct1 =
596163911924388481795636232510643603554710898112024814530157208958563954 3
543496627985540773185452108709958107818159430835510386993354596106366458 8
987655974054612257986150203426400563867571048557098990898168388056314803 2
926412834946522932709072108839454964136634651613300868115581722299435961 6
737681983784274513555455340301061302815102944083173679173923728968671113 9
263762964812983235007744190996826476019779707772600847990363065085978070 2
912227659508058048333611545871333852237218173220807811780955378188955519 1
883178157241590455408910096212697893247529197116309329028589569527960811 3
388386248318556724634385312664 55
```

```
ct2 =
1179205429865439786598365150791228263283147168033431250991894512079786287
6661899077559686851237832931501121869814783150387308320349940383857026679
1418304028077153973323166014396147413152780338536464182756321741608167846
1898274383420499740286693129561920282663362969016442951272395724107242166
3170829944076753483616865208617479794763412611604625495201470161813033934
4768689496126512761043397471652762049451250012747771345294911528406720100
1094003450325731555551127432583168479304020922481687977872561246854275877
7428888563266233284958660088175139114166433501743740034567850893745466521
1443716709621210629920823129487 89
e = 65537

e1 = e
e2 = N + 1

g, u, v = xgcd(e1, e2)

print(g)
print(u*e1 + v*e2 == g)
ct2_inv = inverse(ct2, N)
m = pow(ct1, u, N) * pow(ct2_inv, -v, N) % N
print(m)
print(long_to_bytes(int(m)))
```

运行结果如下：

```
1
True
3176666995451249134434439822293588757184710260394558203398455337887666810874364351965384641608
b'ISCTF{Congratulations_you_master_Mathematical_ability}'
```

小蓝鲨的RSA密文

RSA，又是泄露特定位上的信息，很难让人不认为是考察 **coppersmith** 攻击方法的理解，但是仔细查看过后又并非如此，我们来分析一下代码。

首先，密文的生成逻辑是这样的：

$$c \equiv m^3 + a_2 m^2 + a_1 m + a_0 \mod N$$
$$a_2 = A + n$$

其中，A 是已经泄露的高位信息，n 是未知的低位。所以进一步而言，上述方程简化为：

$$c \equiv m^3 + Am^2 + nm^2 + a_1 m + a_0 \mod N$$

又因为 m, n 是比较小的量，所以显然我们会考虑构造 $f(m,n) \equiv 0 \mod N$ 的一个二元同余多项式，对其使用 **coppersmith** 攻击求解。

但是，正是因为 m, n 实在是太小了，所以我们可以认为这个方程就是一个整数域上的多项式。因此，我们直接遍历 n 的所有取值，直接解出 m 。

具体代码如下:

```
from sage.all import *
from Crypto.Util.number import long_to_bytes
from Crypto.Cipher import AES
from Crypto.Util.Padding import unpad

N =
12128860062119838966224647927763229480042369782336318889666877545677164180723378141652528223478787343590474757146845295047981793568484814365171634360663365696939506558842398244088446454242874286138820030641782222859131670391650417024599042392589447784867949097936492384842664314965975824123990084554453788677 7
c =
37568249853475089675497767737250457730593118393705271492197200840083122471645016882416985628549427563694200034791 17
a2_high = 9012778
a1 = 621315
a0 = 452775142
iv_hex = "bf38e64bb5c1b069a07b7d1d046a9010"
ct_hex =
"8966006c4724faf53883b56a1a8a08ee17b1535e1657c16b3b129ee2d2e389744c943014eb774cd24a5d0f7ad140276fdec72eb985b6de67b8e4674b0bcdc4a5"

iv = bytes.fromhex(iv_hex.strip())
ct = bytes.fromhex(ct_hex.strip())

LOW_BITS = 16

def solve_direct():
    print("-" * 60)
    print("[*] 分析: c 的位数远小于 N，这是一个整数方程。")
    print("[*] 开始遍历 y (0 ~ 65535)...")

    # 定义整数多项式环
    R = PolynomialRing(ZZ, 'x')
    x = R.gen()

    # 预计算常量
    A_high = a2_high << LOW_BITS
    const_part = a0 - c

    # 遍历 y
    # 方程: x^3 + (A_high + y)*x^2 + a1*x + (a0 - c) = 0

    for y_val in range(65536):
        # 构造当前 y 对应的系数
        coeff_x2 = A_high + y_val
```

```python
        # 定义多项式
        f = x**3 + coeff_x2 * x**2 + a1 * x + const_part

        # 求解整数根
        # 由于是单调函数且求解整数根，Sage 的 .roots() 非常快
        roots = f.roots(ring=ZZ, multiplicities=False)

        if roots:
            m_found = roots[0]
            print(f"[+] 找到解! y = {y_val}")
            print(f"[+] m = {m_found}")
            return m_found

    return None

# 执行求解
m_val = solve_direct()

if m_val:
    print("-" * 60)
    print("[*] 尝试解密...")
    try:
        # 恢复 Key
        key = long_to_bytes(int(m_val))
        # 补齐到 16 字节
        key = key.rjust(16, b'\0')

        cipher = AES.new(key, AES.MODE_CBC, iv=iv)
        pt = unpad(cipher.decrypt(ct), 16)

        print(f"[SUCCESS] Flag: {pt.decode()}")
    except Exception as e:
        print(f"[-] 解密失败: {e}")
        # 打印原始 hex 方便调试
        cipher = AES.new(key, AES.MODE_CBC, iv=iv)
        print(f"[-] Raw Decrypted (hex): {cipher.decrypt(ct).hex()}")
else:
    print("[-] 未找到解，请检查参数是否正确。")
```

运行结果如下:

```
------------------------------------------------------------
[*] 分析: c 的位数远小于 N，这是一个整数方程。
[*] 开始遍历 y (0 ~ 65535)...
[+] 找到解! y = 10219
[+] m = 155455820692697783953491152103673430935
------------------------------------------------------------
[*] 尝试解密...
[SUCCESS] Flag: ISCTF{i7_533M5_Lik3_You_R34lLy_UNd3R574nd_Polinomials_4nD_RSA}
```

baby_math

我们还是关注一下密文的生成过程，过程如下：

$$c = a\sin(x) + b\cos(x)$$

其中，a, b 各是我们要求解的明文的一部分，显然我们会考虑构造格进行求解，构造如下的矩阵 M 。

$$M = \begin{pmatrix} 1 & 0 & K\sin(x) \\ 0 & 1 & K\cos(x) \\ 0 & 0 & Kc \end{pmatrix}$$

矩阵 M 的每行是格的基向量，K 是缩放因子，将有理数转为整数。我们考虑这样的线性组合 $(a, b, -1)$ ，那么组合后得到：

$$(a, b, K(a\sin(x) + b\cos(x) - c)) \approx (a, b, 0)$$

> **Note**
>
> 为什么我们使用约等号，因为计算机计算浮点数的时候是有精度损失的。

因为组合得到的向量存在分量为 0 ，而 a, b 相对于 K 来说是非常小的（a, b 约为 Flag 的一半长度，大概几百比特，而 K 可以设得很大）。因此，向量 $(a, b, 0)$ 将是格中的一个**短向量**，进而我们使用格基规约求解。

具体代码如下：

```
from Crypto.Util.number import long_to_bytes
from sage.all import *

R = RealField(1000)

x_val =
"0.75872961153339387563860550178464795474547887323678173252494265684893323
6546066286514271518668187301003575902968632742367190736846200307171415219
4121116728217056742411427094154201613597943827143904719402894399750812638
9603529160316379547558098144713802870753946485296790294770557302303874143
106908193100"
x = R(x_val)

enc_val =
"1.2483997840872858018118302767578598278476482159215689259813600036339726
7152291738689909414790691435938223032351375697399608345468567445269769342
3003251922484380389639772072962419712179551784431705986296484147063452167
9704337440854120316771939681892595380138762388420090170360628866414137504
9626635852e52"
enc = R(enc_val)

# 缩放因子 K，需要足够大以保留精度，同时覆盖 a 和 b 的大小
```

```python
K = 2**512

# 计算基向量的系数
k_cos = int(cos(x) * K)
k_sin = int(sin(x) * K)
k_enc = int(enc * K)

# 构造矩阵
# 目标是寻找线性组合：a*row1 + b*row2 - 1*row3 \approx (a, b, 0)
M = Matrix(ZZ, [
    [1, 0, k_cos],
    [0, 1, k_sin],
    [0, 0, k_enc]
])

print("Running LLL...")
res = M.LLL()

# 3. 提取结果
for row in res:
    # LLL 结果的符号可能反转，取绝对值
    a_cand = abs(int(row[0]))
    b_cand = abs(int(row[1]))

    # 简单的过滤：a 和 b 不应该为 0
    if a_cand == 0 or b_cand == 0:
        continue

    try:
        part1 = long_to_bytes(a_cand)
        part2 = long_to_bytes(b_cand)

        flag = part1 + part2

        # 尝试解码并打印
        # 检查是否看起来像 flag
        if b'ISCTF' in flag:
            print(a_cand)
            print(b_cand)
            print(f"[+] Flag Found: {flag.decode()}")
            break
        else:
            pass

    except Exception as e:
        continue
```

运行截图如下：

```
Running LLL...
107164967250478127778884915227180531418197665393972
180323050773397726623520934949177411058104504843769 57
[+] Flag Found: ISCTF{164a3221-7306-4024-88c3-4ef557b86895}
```

小蓝鲨的 LSFR 系统

题目的输出中已经给出了**初始状态**和移位生成的**输出状态**，题目中的密钥是通过掩码生成的，所以我们把目光看到掩码上来：

```
feedback = sum(state[i] & mask[i] for i in range(128)) % 2
```

这个操作还是很常见的，我们可以将其看为 $GF(2)$ 上的线性运算，掩码共有 128 位未知，而循环生成有 256 轮，这是一个超定方程组，剩下的就是写代码求解了。

具体代码如下：

```
import binascii
from sage.all import *

# 1. 输入数据
# 题目给出的初始状态 (128位)
init_state = [0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0,
1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 1, 0, 1,
1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 1, 0,
1, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1,
1, 0, 1, 0, 1, 0, 0, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1,
0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0]

# 题目给出的输出状态 (256位)
output_state = [0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 1,
1, 1, 1, 0, 1, 1, 1, 0, 0, 0, 1, 1, 0, 0, 1, 0, 0, 0, 1, 0, 1, 1, 1, 1,
1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0, 0,
0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 0,
0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1,
0, 1, 1, 0, 1, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 1,
0, 0, 0, 1, 1, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1,
0, 0, 1, 1, 0, 0, 1, 1, 1, 0, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0,
1, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 0, 0, 1, 0, 1, 1, 1, 0,
1, 1, 0, 0, 1, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1,
1, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 0, 0, 1]

ciphertext_hex = '4b3be165a0a0edd67ca8f143884826725107fd42d6a6'

# 2. 准备数据流
# 完整的 LFSR 滑动窗口数据流 = 初始状态 + 输出状态的前半部分
# 因为 output_state[0] 是由 init_state 产生的
# output_state[1] 是由 init_state[1:] + [output_state[0]] 产生的
```

```
full_stream = init_state + output_state

# 3. 构建方程组 A * Mask = B
F = GF(2)
num_vars = 128
num_eqs = 256  # 我们可以利用全部 256 个输出来构建方程，保证准确性

# 矩阵 A: 每一行是某一时刻的 128 位状态
A = Matrix(F, num_eqs, num_vars)
# 向量 B: 每一位是该时刻产生的 feedback (即 output_state)
B = vector(F, num_eqs)

print("[*] 构建方程组...")
for i in range(num_eqs):
    # 取当前时刻的窗口，长度 128
    window = full_stream[i : i + num_vars]
    A.set_row(i, window)

    # 结果就是对应的 output_state[i]
    B[i] = output_state[i]

# 4. 求解 Mask
print("[*] 正在求解 Mask...")
try:
    # 求解 A * x = B
    mask_vec = A.solve_right(B)
    mask = [int(x) for x in mask_vec]
    print("[+] 成功解出 Mask!")

    # 5. 生成 Key 并解密
    # 题目逻辑: key = bytes(int(''.join(str(bit) for bit in mask[i*8:
(i+1)*8]), 2) for i in range(16))
    key_bytes = []
    for i in range(16):
        byte_bits = mask[i*8 : (i+1)*8]
        byte_val = int(''.join(str(b) for b in byte_bits), 2)
        key_bytes.append(byte_val)

    key = bytes(key_bytes)
    print(f"[+] Key (hex): {binascii.hexlify(key)}")

    # 解密 Ciphertext
    cipher = binascii.unhexlify(ciphertext_hex)

    # 异或解密
    # keystream 循环使用 key
    # keystream = (key * (len(plaintext)//16 + 1))[:len(plaintext)]
    keystream = (key * (len(cipher)//16 + 1))[:len(cipher)]

    plaintext = bytes([c ^ k for c, k in zip(cipher, keystream)])
```

```
    print("-" * 40)
    print(f"[+] FLAG: {plaintext}")
    print("-" * 40)

except ValueError:
    print("[-] 无解。")
```

运行截图如下：

```
[*] 构建方程组...
[*] 正在求解 Mask...
[+] 成功解出 Mask!
[+] Key (hex): b'0268a231e6db81b049faae29dd3b522d'
----------------------------------------
[+] FLAG: b'ISCTF{lf5R_jUst_So_s0}'
----------------------------------------
```

> ⚠ **Warning**
>
> emm，值得注意的问题是题目的代码好像存在着一些问题，请看到这几行代码。
>
> ```
> for _ in range(256):
>  feedback = sum(state[i] & mask[i] for i in range(128)) % 2
>  state.append(feedback)
> ```
>
> 每次循环都是求解状态列表的前 128 位，但是题目并没有做到旧状态的弹出，这意味着在 256 次循环生成的状态应该是全部一致的，这就是矛盾的地方了。

小蓝鲨的密码箱

打开题目发现是没怎么接触过的黑盒密码，那我们只能尝试带入一些特殊值分析加密算法了。

首先当然是尝试最特殊的 0, 1, -1 这样的数了，发现不允许参数为 0 ，那我们接着尝试 1 。加密的明文我们选取单个字符，然后观察密文的 16 进制表示，单个字符比较好看出加密过程发生的变化。此外，我们发现当 $c = 1$ 时，不管明文是什么，输出的密文总为 0 。结合数学知识想一下，我们推测 c 是模数，当模为 1 的时候，任何数对 1 取模都等于 0 。

因此，我们可以输入一个非常大的 c 值，从而忽略模数的影响。然后，我们得到如下结果。

a

1

b

1

c

99999999999999999999999999999999

What do you want to encrypt?:

1

加密

加密结果

原文: 1

密文:

```
32
```

Flag:

```
4a 54 44 55 47 7c 66 34 35 65 34 66 66 37 2e 39 31 64 31 2e 35 37 33 34 2e 63 39 67 37 2e 63 67 32 39
63 64 36 39 32 33 67 31 7e
```

使用的参数: a=1, b=1, c=99999999999999999999999999999999

而字符 1 的 **ASCII** 码的十六进制表示就是 0x31 ，与加密后相比就是少了 1 ，所以我们推测加密逻辑就是对每个字符的十六进制加一然后输出十六进制。

最后，我们可以给出代码如下：

```python
cipher = "4a 54 44 55 47 7c 66 34 35 65 34 66 66 37 2e 39 31 64 31 2e 35
37 33 34 2e 63 39 67 37 2e 63 67 32 39 63 64 36 39 32 33 67 31 7e"

# 转为字节列表
bytes_list = [int(b, 16) for b in cipher.split()]

# 解密：每个字节 -1
plain_bytes = [(b - 1) & 0xFF for b in bytes_list]

# 转为字符串
result = ''.join(chr(b) for b in plain_bytes)
print(result)
```

运行后得到 flag: ***ISCTF{e34d3ee6-80c0-4623-b8f6-bf18bc5812f0}***

沉迷数学的小蓝鲨

这场比赛最谜语人的题目来了，包括出题人也承认了这题受到了比较多师傅的吐槽（）。首先根据广泛用于区块链的提示，我们知道了这条曲线是 **Secp256k1**，进而我们确定了p, a, G这三个参数。接着我们就会发现G, Q这两个点不在题目给出的这条曲线上，所以这道题实际考察的是**无效曲线攻击（Invalid Curve Attack）**。

这个攻击的简要思想是这样的，椭圆曲线加密的关键在于计算点的乘法，也就是$Q = kP$。在计算乘法的过程就是不断地计算切线，然后求解交点，最后对称获得。又因为椭圆曲线一般形式如下：

$$y^2 = x^3 + ax^2 + b$$

所以曲线上任意一点的切线斜率为：

$$2y\frac{\mathrm{d}y}{\mathrm{d}x} = 3x^2 + 2ax$$

从上式可以看出求解切线斜率与b是没有关系的，因此如果攻击者发送一个点P'，这个点并不满足原来的曲线方程（即它属于另一个b'不同的曲线），受害者的计算程序如果没有检查机制，依然会照常套用公式进行计算。程序虽然在“跑”，但实际上是在另一条参数为(a, b')的曲线上运行。

而我们作为攻击者会特意选择这个b'，使得这条新曲线E'的阶包含很多小的素数因子，这使得在该曲线上解决离散对数问题变得极其容易。

具体代码如下：

```
from sage.all import *
from hashlib import *

# 1. 定义题目给出的参数
p = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEFFFFFC2F
a = 3
b = 27

# 题目给出的坐标
Qx = 0xa61ae2f42348f8b84e4b8271ee8ce3f19d7760330ef6a5f6ec992430dccdc167
Qy = 0x8a3ceb15b94ee7c6ce435147f31ca8028d1dd07a986711966980f7de20490080

Gx = 0x79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798
Gy = 0x483ADA7726A3C4655DA4FBFC0E1108A8FD17B448A68554199C47D08FFB10D4B8

F = GF(p)
print(Qy**2 % p == (Qx**3 + a*Qx + b) % p)
print(Gy**2 % p == (Gx**3 + a*Gx + b) % p)

b_implicit = (F(Gy)**2 - F(Gx)**3 - 3*F(Gx))
print(f"[*] Checking implicit curve with b = {b_implicit}")
E_implicit = EllipticCurve(F, [a, b_implicit])
```

```
if E_implicit(Qx, Qy):
    print("[!] Found! Both G and Q are on the implicit curve.")
    target_curve = E_implicit
    base_point = E_implicit(Gx, Gy)
    target_point = E_implicit(Qx, Qy)
else:
    print("[-] Q is not on the implicit curve derived from G.")
    # 这种情况下，我们仅使用 b=27 的曲线进行离散对数求解
    # 注意：如果 G 真的不在曲线上，常规方法无法直接解 Q = k*G。

# 无效曲线攻击的关键：这个构造出的曲线的阶通常包含很多小因子
order = target_curve.order()
print(f"[*] Curve Order: {order}")

factors = list(factor(order))
print(f"[*] Factors: {factors}")

# Sage 的 discrete_log 会自动识别光滑阶并使用 Pohlig-Hellman 算法
print("[*] Solving Discrete Logarithm (this may take a few seconds)...")
k = discrete_log(target_point, base_point, operation='+')

print("-" * 30)
print(f"Found private key k: {k}")
k_hex = format(k, 'x')
flag = md5(k_hex.encode()).hexdigest()
print("ISCTF{" + flag +"}")
print("-" * 30)
```

运行后得到结果：

```
False
False
[*] Checking implicit curve with b = 66385389407800359838405813331870212727788158969948345552414605934650319155613
[!] Found! Both G and Q are on the implicit curve.
[*] Curve Order: 115792089237316195423570985008687907853081925042010425286133534915827977481598
[*] Factors: [(2, 1), (3, 2), (7, 1), (53, 1), (4733, 1), (17669, 1), (42437, 1), (48858409517673714640067242544283991386
[*] Solving Discrete Logarithm (this may take a few seconds)...
------------------------------
Found private key k: 954761
ISCTF{43896099feea21a3d5804863075e1aaa}
------------------------------
```

小蓝鲨的费马谜题

题目都提到了费马，那显然我们要用到费马小定理了，我们做如下的数学推导。

首先我们需要分析 `hint_value` 的数学构成。代码中给出：

$$h_{i,j} \equiv b_i^{p-1} + b_j^{p-1} \pmod{n}$$

其中 b_i, b_j 是很小的素数（100以内），而 p 是一个 1024 位的大素数。显然 $\gcd(b_i, p) = 1$ 。

根据**费马小定理**：如果 p 是素数，且 a 是不可被 p 整除的整数，那么：

$$a^{p-1} \equiv 1 \pmod{p}$$

我们可以将 $h_{i,j}$ 对 p 取模来观察其性质：

$$\begin{aligned} h_{i,j} \pmod{p} &= (b_i^{p-1} + b_j^{p-1}) \pmod{p} \\ &= (b_i^{p-1} \pmod{p} + b_j^{p-1} \pmod{p}) \pmod{p} \\ &= (1+1) \pmod{p} \\ &= 2 \pmod{p} \end{aligned}$$

这意味着：

$$h_{i,j} - 2 = k \cdot p$$

即 $h_{i,j} - 2$ 是 p 的倍数。因此，p 是 $n, h_{i,j} - 2$的最大公因数，进而我们便可分解得到私钥。

具体代码如下：

```
from Crypto.Util.number import *

n =
169267471837308114455211822876318710952358071246373250966607593619961553699399987456382938627262677418908406540947940276001775649488193720309330792910970841770918639857492407560852436544423747228825070153435158277871413079091828200133540703217384058102571076518577396070602745494126925171402597173461705249205408880503230669881088369119754666030730344338318872089781304067427143029402647028743050956026233791773538733472087517210684986909179327769841905981437045676654751614533356296592007487866482883094015138567403234559469013129888412909176667320777474570813558537228321663315017796011577197222915987877107469179 47
e = 65537
c =
713566988850899328399888725752618581383178020868078833333204493034212538156191983008408863192030162390994944300207319338140176190139882671966541143201621740045761354530826283197556445623116511409190474880820633048823156977316274569660236646875366418826193301419821892245971597287674095726013224392754903784026575328253456567428090843987555017980178871173790163234913678058400759965505560577265112700371113851299868314576374383932646031944018609981850707843327129168519494425479569042432719262525870183565463983228540299099566284642656178950833179997232971141021780265768284238210586944685320763407029595928137548493 3

parsed_data = []
with open('output.txt', 'r') as f:
    for line in f:
        line = line.strip()
        if not line or 'Hint' not in line:
```

```
            continue

        # 步骤 1: 通过冒号 ':' 分割，去掉 "Hint X" 部分
        # 例如: "Hint 3: 59, 73, 69..." -> ["Hint 3", " 59, 73, 69..."]
        parts = line.split(':')
        if len(parts) < 2:
            continue

        content = parts[1].strip()
        # 步骤 2: 通过逗号 ',' 分割数值部分
        # 例如: "59, 73, 69..." -> ["59", " 73", " 69..."]
        nums = content.split(',')

        if len(nums) == 3:
            base1 = int(nums[0].strip())
            base2 = int(nums[1].strip())
            hint_value = int(nums[2].strip())
            parsed_data.append((base1, base2, hint_value))

print(f"[*] 成功解析 {len(parsed_data)} 条 hint。")

for i in parsed_data:
    if GCD(n, i[2] - 2) != 1:
        p = GCD(n, i[2] - 2)
        break

q = n // p
phi = (p - 1) * (q - 1)
d = inverse(e, phi)
m = pow(c, d, n)
print(long_to_bytes(m))
```

运行后得到 flag：***ISCTF{ModIFi3D_f3RM47_7H3or3m_I5_fUn_8U7_h4rD3r!}***

小蓝鲨的后门

emm，很典型的一个 RSA 素数生成存在漏洞的问题，当私钥的选取满足如下形式时：

$$4p - 1 = D \times V^2$$

其中，D 是给定的某个整数，V 是随机生成的 2048 bits的正整数。对于这种满足 $4p - 1$ 形式的质数等式，我们可以利用基于**复杂乘法 / 特殊构造椭圆曲线 + n 次除法多项式**的方法来暴露 p 。

原作者及其代码链接：https://github.com/pwang00/Cryptographic-Attacks/tree/master/Public%20Key/Factoring

由于这个标准脚本每次只是针对一个 D 进行尝试分解，所以这道题我们直接多试几次，最后可以确定具体的值。

从而得到具体代码如下:

```
import sys
from utils import *
from Crypto.Util.number import inverse, long_to_bytes

sys.setrecursionlimit(100000000)

def cm_factor(N, D, B=32, debug=True):
    """
    Implements a simplified version of Cheng's 4p - 1 elliptic curve
complex multiplication based factorization algorithm.
    Targets moduli where one (or many) prime factors satisfies the
equality 4p_i - 1 = D * s^2, where D is a squarefree integer
    and s is a randomly generated one between an upper and lower bound.

    Original paper title: I want to break square-free: The 4p - 1
factorization method and its RSA backdoor viability
    Link: https://crocs.fi.muni.cz/_media/public/papers/2019-secrypt-
sedlacek.pdf

    :param N: integer to be factored
    :param D: squarefree integer satisfying 4p - 1 = D * s^2
    :param B: number of iterations to run the algorithm for
    :param debug: switches debugging information on/off
    :return: a tuple corresponding to p, q, or failure (-1)
    """

    # If D is not squarefree then we terminate immediately.
    assert D.is_squarefree(), "D must be squarefree."

    # Computes the -Dth Hilbert polynomial modulo N and quotient ring Q =
Z_N[x] / <H_{-D, N}>

    Z_N = Zmod(N)
    P = Z_N[x]
    H = P(hilbert_class_polynomial(-D))
    Q = P.quotient_ring(H)

    # j is the equivalence class corresponding to [X] in Q.
    j = Q(x)

    # The paper claims that we can treat both 1728 - j and H as
polynomials in Z_N[X] and calculate the inverse of
    # 1728 - j and H via egcd. This doesn't quite work off the shelves,
so we instead accomplish this by treating
    # 1728 - j as an element of Q = Z_N[x] / <H_{-D, n}> and lifting it
back into the base ring.
```

```
    # Sage implements this via the .lift() method.

    if debug:
        print("Calculating inverse mod of 1728 - j with H...")

    try:
        k = j * polynomial_inv_mod((1728 - j).lift(), H)
    except ValueError as e:
        r = gcd(int(e.args[1].lc()), N)
        return int(r), int(N // r)

    # Constructs an elliptic curve described by the equation y^2 = x^3 +
ax + b where a = 3k and b = 2k over Q.
    # This is so we can calculate the division polynomial psi_n(a, b,
x_i) later.
    E = EllipticCurve(Q, [3*k, 2*k])

    for i in range(B):

        # Obtains a random element from Z_N for the division polynomial.
        x_i = Z_N.random_element()

        if debug:
            print("Calculating division polynomial with x_i...")

        # Calculates the division polynomial modulo n: psi_n(a, b, x_i)
        z = E.division_polynomial(N, x=Q(x_i))

        # If our egcd implementation throws an exception, then we know
that the r polynomial during the computation
        # has no inverse; we return this polynomial in our exception and
then take the gcd of its leading coefficient with N.
        # Otherwise we take the gcd of d and N normally.
        try:
            d, _, _ = polynomial_egcd(z.lift(), H)
        except ValueError as e:
            r = gcd(int(e.args[1].lc()), N)
            return int(r), int(N // r)

        r = gcd(int(d), N)

        # Ensure that the factorization is nontrivial: i.e. r != 1 and r
!= N
        if 1 < r < N:
            return r, N // r

    return -1

if __name__ == "__main__":
    D = 427
```

```
    N =
3309617528879961733288549359651125888844035845310225611197437406503649582
2068464075458439385079681283300784394033678459971922442896911953328428642
4077547165101460469847980799370419655082069179038497637761333327079374599
5065747238921438172267518068026760132251884674032746582115636558765009972
9691742190461412805684797779814685533693930646305944041615049326297326943
1000762285579221126342017624118238829230679953011897314722801993750454924
6270742643536920600027585214015443613852313543139818360568555829296708112
5911301901297054082495113948914639318253241487821418208699929839737784553
4568556100933934481180701997394558264969597606662342898026915506749002491
3262507921073481766817959427999545260685014991002325986586501845658732435
2517683345166425491765570317847294474465862853419534697702341855076162025
4528178516972066618936960223660362493931786389085393392950207048675797593
8162714357001309952254833166258361048026081637453766338848405885753559367
4617306865531964557210014951552413188381377348691712215324849502237269091
2572541775943614626733948206252900473118240712831444072243770979419529210
0348839031110384483669333748415311264214412320245144861687426862974810630
8916197705482562109976865909750993940531505632533612092949283847930960995
8696957890570295444494277819063443427972643459784894450787015151715676537
3852377679904067425476643215636888292898093215347522442605293194543165325
8041618243874984992335406012522932804396135589408657623851913886829849924
9023773237770103057707912709725417033309061308880583988666463892828633292
8399688669537769897223109542045507838257047100174342146441994157565849292
1423967943321139323030778295306724652962613644631494125887743935609477533
7541321331600788042698664632064112896956898222397445497695982546922871549
8282429383684867746173504207907110930699109141353196353307862533312234596
3723210641757722535044129 1
    print("Factoring...")
    r, s = cm_factor(N, D)
    assert r * s == N
    print(f"Factorization successful!")
    print(f"Successfully find r and s, try to solve RSA")
```

```
    c =
212346990585867204560749978406244213251144548138396149876252930813225573964743743310623240076921482426544897728053124373315032929324133875314957266495139484699567528919690660711722222757624739519047423652631612556832113937491418819741530730836305579152302101616418756366798409439714043836489654997308261399789368847924248898076302503602653274829239053799622592966934018805688066167076708873399729510077548471187650270072877991644415706656886220388746860133005018914685022732719262919710874193422600082385583629822405629220392922887898181915985227173881016604498106652501844316026711412956745976171977119703328814921559308363060138203377963970984712220593336713159734968921097855310094948675421549640099769461209367374396482468152129218666438977024522653691528013528015440313034981983973685977844334196152779256456715583345981234311241518377392361450912536318491335347506959721906982903993728410239088038142803971132404444984691815547201058733797833802388895579267131452325576042575399419473847790510049088515795639055741434593740667515359201087211978944672386143676684477951410239359787877543382974142784694266278769539348943087451443086676857453247653232890469797970446429750318117491749329986347358826779764098893564081668934172047327871380355612977733499211711098892738572628861337162120664600564875970420208200656165153975734830572375804620405804831699449133408497126664753227381291735116991284383676729731736719032544070913866946805107500261134522561008047413499767639218832012703609787581477514089093664923903581385136180130749154323775598371241866152815888072763101593178296658155000487023879474002640710892103951717343305638528289523819399179422853068955397100518956727546826783306797955126590186360647747653520107951377821594811086096453984438199315753748764774337233049084739849310994774328174885402775747471641470284242744811928033021628049
    e = 65537
    phi = (r-1)*(s-1)
    d = inverse(e, phi)
    m = pow(c, d, N)
    print(long_to_bytes(int(m)))
```

运行得到结果:

```
Factoring...
Calculating inverse mod of 1728 - j with H...
Calculating division polynomial with x_i...
Factorization successful!
Successfully find r and s, try to solve RSA
b'ISCTF{You_have_an_extraordinary_talent_in_cryptography!_But_do_you_understand_number_theory_well_enough?}'
```